Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

Apache Iceberg: transactional tables on S3

Setting: Transactional data (orders, payments, users) lives in an OLTP database (PostgreSQL, MySQL) where rows are constantly **inserted, updated and deleted**. For analytics we want that data in a cheap data lake on **Amazon S3**, stored as Parquet files. The problem: S3 is an object store, not a database. It has no transactions, no `UPDATE`, no `DELETE`, and files are immutable.

Apache Iceberg is an **open table format**: a specification of metadata files that sits on top of Parquet files in S3 and makes a set of files behave like a **database table**, with ACID transactions, updates, deletes, schema changes and time travel. It is not a query engine and not a storage system: engines (Spark, Flink, Trino, Athena, Snowflake, DuckDB) read and write it, and S3 stores it.

$$\text{Lakehouse} = \underbrace{\text{S3}}_{\text{storage}} + \underbrace{\text{Parquet}}_{\text{file format}} + \underbrace{\text{Iceberg}}_{\text{table format}} + \underbrace{\text{Catalog}}_{\text{table registry}} + \underbrace{\text{Spark / Trino / } \ldots}_{\text{compute}}$$

Before going into Iceberg, two concepts it relies on: **OLTP vs OLAP** (where the data comes from and where it goes) and **ACID** (the guarantee Iceberg adds on top of S3).

Background 1: OLTP vs OLAP

OLTP (Online Transaction Processing) is the database that *runs* the business: every purchase, payment, login or status change is a small transaction that reads or writes a few rows. **OLAP (Online Analytical Processing)** is the system that *analyzes* the business: few queries, but each one scans and aggregates millions of rows.

	OLTP	**OLAP**
Typical question	"What is the status of order 123?"	"How much did we sell per region in 2026?"
Operations	Many small `INSERT` / `UPDATE` / `DELETE`, few rows each	Few large `SELECT` with `GROUP BY`, millions of rows each
Users	Applications, thousands of concurrent users	Analysts, data scientists, BI dashboards, ML pipelines
Data	Current state, normalized (many joined tables)	History, often denormalized (star schema, wide tables)
Storage layout	Row-oriented	Column-oriented (Parquet, ORC)
Priority	Low latency per transaction, strict ACID	High throughput per query, cheap storage
Examples	PostgreSQL, MySQL, Oracle, SQL Server, Aurora	Snowflake, BigQuery, Redshift, Trino / Athena + Iceberg on S3

Why the storage layout matters: consider a table with columns `id`, `customer`, `amount`, `date`.

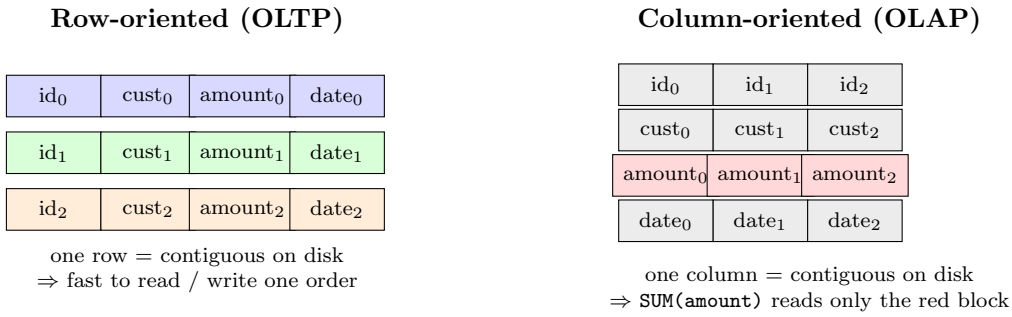


Figure 1: Same table, two layouts. OLTP optimizes access to a full row; OLAP optimizes scanning a few columns over many rows.

If a table has c columns and a query uses only c_q of them, a columnar scan reads roughly a fraction $\frac{c_q}{c}$ of the data (even less after compression, since values of the same column are similar).

Why this matters for Iceberg: running heavy analytical queries directly on the production OLTP database would slow it down for real users. The standard solution is to **copy the changes** of the OLTP database (via CDC) into S3 as Parquet, organized as Iceberg tables, and run all OLAP workloads there. Iceberg is what lets those copies receive the OLTP updates and deletes correctly.

Background 2: ACID

A **transaction** is a group of operations that must behave as a single unit. Classic example: transfer $100 from account A to account B.

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 'A';
UPDATE accounts SET balance = balance + 100 WHERE id = 'B';
COMMIT;
```

ACID is the set of four guarantees that make transactions reliable:

- **A, Atomicity:** all or nothing. If the system crashes after debiting A but before crediting B, everything is undone; the $100 never disappears.
- **C, Consistency:** a transaction moves the database from one valid state to another, respecting the declared rules (e.g. $balance \geq 0$, unique keys, foreign keys). If a rule would be violated, the transaction is rejected. For the transfer, the total money is preserved:

$$balance_A + balance_B \text{ (before)} \;=\; balance_A + balance_B \text{ (after)}$$

- **I, Isolation:** concurrent transactions do not interfere. Another user querying during the transfer sees the state *before* or *after* it, never the intermediate state with A debited and B not yet credited. Formally, the result is as if transactions had run one after the other (serializable), or a weaker but well-defined level such as **snapshot isolation**: each transaction reads a fixed snapshot of the data.
- **D, Durability:** once the database confirms the `COMMIT`, the change survives crashes and power failures (it is persisted to disk / replicated storage).

Where ACID holds and where it does not:

System	ACID?	Why
OLTP database (PostgreSQL, MySQL)	Yes	Transaction log (WAL), locks / MVCC, built for it
Plain Parquet folders on S3	No	A job writing 100 files that fails at file 60 leaves partial data visible; readers can mix old and new files
Iceberg tables on S3	Yes	**A:** the catalog pointer swaps atomically. **C:** conflicts are validated before commit. **I:** every reader is pinned to one snapshot. **D:** all files and metadata are in S3

In short: the OLTP source is ACID, plain S3 is not, and **Iceberg restores ACID on S3**, so the analytical copy of transactional data is as trustworthy as the original.

Why not just Parquet folders (Hive-style tables)?

The classic data lake defines a table as "all files inside `s3://bucket/orders/`". This causes:

(a) **No atomicity:** a job that writes 100 files and fails at file 60 leaves a half-written table that readers will see.

(b) **No isolation:** a reader listing the folder while a writer is replacing files may read old and new files mixed.

(c) **Slow planning:** to know which files form the table, the engine must *list* S3 prefixes, which is slow with thousands of partitions.

(d) **No row-level changes:** to update one row, you must rewrite the whole partition by hand.

(e) **Fragile schema and partitions:** renaming a column or changing the partitioning means rewriting all the data.

Iceberg's key idea: **the table is not a folder, it is an explicit list of files** tracked in metadata. A file belongs to the table only if the current metadata says so.

Architecture: three layers

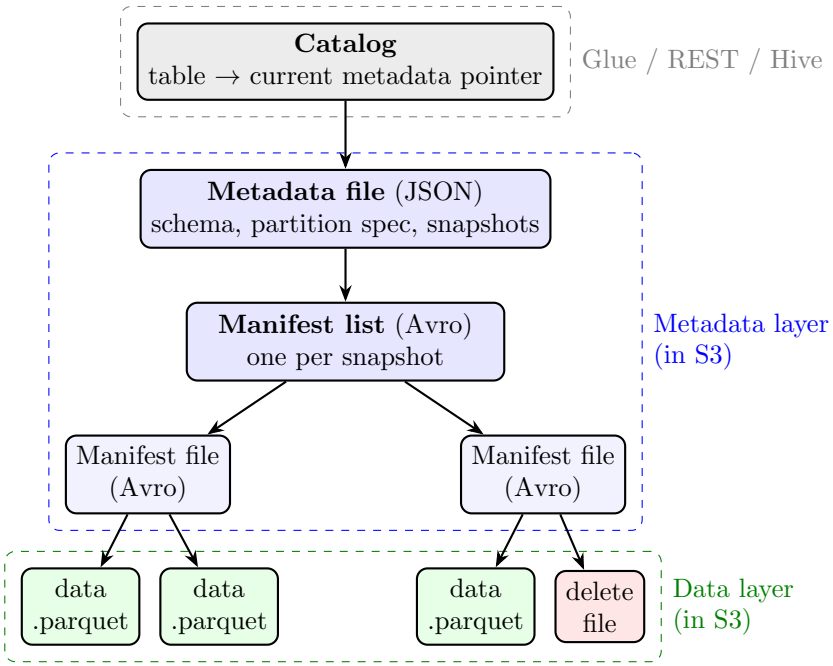


Figure 2: The catalog points to the current metadata file; everything else is immutable files in S3.

- **Catalog:** maps a table name (`db.orders`) to the location of its *current* metadata file. Its only critical job is to swap that pointer atomically. Options: AWS Glue Data Catalog, Iceberg REST catalogs (Apache Polaris, Project Nessie, Unity Catalog, Amazon S3 Tables), Hive Metastore.

- **Metadata file:** schema (with column IDs), partition spec, sort order, and the list of all **snapshots**. Each commit writes a new one.
- **Manifest list:** one per snapshot; lists the manifest files of that snapshot, with partition ranges for each.
- **Manifest files:** list data files and delete files, with per-file statistics: row count, and min / max / null count per column.
- **Data files:** Parquet (or ORC / Avro). **Delete files:** mark rows as deleted without rewriting data.

Planning without listing: thanks to the statistics, a query filtering `order_date = '2026-11-01'` prunes manifests by partition range and then files by min / max, **without listing S3**. Only the files that can contain matching rows are read.

Snapshots: the table as a sequence of immutable states

Let F_k be the set of files that form the table at snapshot k. A commit that adds files A and removes files R creates a new snapshot:

$$F_{k+1} = (F_k \setminus R) \cup A$$

Old files are never modified, so $F_0, F_1, \ldots, F_k$ all remain readable. This gives **time travel** (query the table as of snapshot k or a timestamp) and **rollback** (set the current pointer back to F_k).

ACID with optimistic concurrency

Each writer works without locks:

(1) Read the current metadata version m_{base}.

(2) Write new data files and new metadata m_{new} to S3 (invisible to readers: nobody points to them yet).

(3) Ask the catalog for an atomic **compare-and-swap**:

$$\text{commit succeeds} \iff \text{pointer} = m_{base} \ \text{ at commit time, then } \ \text{pointer} \leftarrow m_{new}$$

If another writer committed first, the swap fails; the writer re-reads the new state, checks for conflicts (did they touch the same files?) and retries. Properties:

- **Atomicity:** readers see either all of m_{new} or none of it; a failed job leaves only orphan files, never a broken table.
- **Isolation:** a reader is pinned to one snapshot for the whole query (snapshot isolation), even if writers commit meanwhile.
- **Durability:** everything is in S3.

Row-level updates and deletes (key for transactional data)

Since Parquet files are immutable, an `UPDATE` or `DELETE` has two strategies:

	Copy-on-Write (CoW)	**Merge-on-Read (MoR)**
How	Rewrite every data file that contains a changed row	Write small *delete files* (position deletes, equality deletes, or deletion vectors in spec v3) plus new rows
Writes	Slow (rewrites whole files)	Fast
Reads	Fast (clean files)	Slower (engine merges deletes at read time)
Best for	Batch, few updates, read-heavy	Streaming CDC, frequent updates

With MoR, periodic **compaction** applies the delete files and rewrites clean data files, recovering read speed.

Schema and partition evolution

- **Schema evolution:** columns are tracked by a unique **ID**, not by name or position. Adding, dropping, renaming or reordering a column is a metadata-only change; no data is rewritten and old files are still read correctly.
- **Hidden partitioning:** the table is partitioned by a *transform* of a column, for example `days(created_at)`. Users filter on `created_at` directly and Iceberg applies the pruning; they never need to know a `created_date` partition column exists.
- **Partition evolution:** the partition spec can change (e.g. from `months` to `days` when the table grows). Old files keep the old spec, new files use the new one, and queries plan correctly over both.

Typical pipeline: transactional database → Iceberg on S3

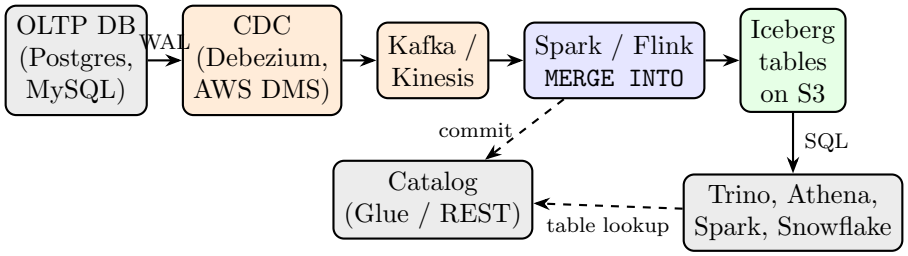


Figure 3: CDC reads the database log (WAL / binlog) and streams inserts, updates and deletes into Iceberg.

(1) **Capture:** Change Data Capture reads the database transaction log, so every `INSERT`, `UPDATE` and `DELETE` becomes an event, without loading the production DB with queries.

(2) **Transport:** events go through Kafka / Kinesis (or land as raw files in S3, the *bronze* layer).

(3) **Apply:** Spark or Flink applies each micro-batch with `MERGE INTO` on the primary key, keeping the latest version of each row (the *silver* layer). Each micro-batch is one atomic Iceberg commit.

(4) **Serve:** any engine reads the same tables via the catalog: Athena / Trino for SQL, Spark for ML features, BI tools on top.

Main operations (Spark SQL)

```
-- Create a table with hidden partitioning
CREATE TABLE lake.sales.orders (
    order_id    BIGINT,
    customer_id BIGINT,
    status      STRING,
    amount      DECIMAL(12,2),
    created_at  TIMESTAMP,
    updated_at  TIMESTAMP
) USING iceberg
PARTITIONED BY (days(created_at))
TBLPROPERTIES ('write.merge.mode' = 'merge-on-read');

-- Upsert a CDC micro-batch (op = 'I', 'U' or 'D')
MERGE INTO lake.sales.orders t
USING cdc_batch s
ON t.order_id = s.order_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED AND s.updated_at > t.updated_at THEN UPDATE SET *
WHEN NOT MATCHED AND s.op <> 'D' THEN INSERT *;

-- Time travel
SELECT * FROM lake.sales.orders TIMESTAMP AS OF '2026-10-31 23:59:59';
SELECT * FROM lake.sales.orders VERSION AS OF 8744736658442914487;

-- Inspect the history (metadata tables)
SELECT * FROM lake.sales.orders.snapshots;
SELECT * FROM lake.sales.orders.files;

-- Schema and partition evolution (metadata only)
ALTER TABLE lake.sales.orders ADD COLUMN channel STRING;
ALTER TABLE lake.sales.orders RENAME COLUMN amount TO total_amount;
ALTER TABLE lake.sales.orders ADD PARTITION FIELD bucket(16, customer_id);
```

Table maintenance (mandatory in production)

Every commit adds files and snapshots; without maintenance, a streaming table accumulates thousands of small files and metadata, and queries get slower over time.

```
-- 1. Compaction: merge small files and apply delete files
CALL lake.system.rewrite_data_files(table => 'sales.orders');
-- 2. Expire old snapshots (limits time travel, frees storage)
CALL lake.system.expire_snapshots(
    table => 'sales.orders',
```

```
    older_than => TIMESTAMP '2026-10-25 00:00:00',
    retain_last => 10);
-- 3. Delete files no snapshot references (e.g. from failed jobs)
CALL lake.system.remove_orphan_files(table => 'sales.orders');
-- 4. Reorganize manifests for faster planning
CALL lake.system.rewrite_manifests('sales.orders');
```

Managed options such as Amazon S3 Tables or AWS Glue table optimizers can run compaction and snapshot expiration automatically.

Iceberg vs the other open table formats

	Apache Iceberg	**Delta Lake**	**Apache Hudi**
Origin	Netflix	Databricks	Uber
Metadata	Tree: metadata file → manifest list → manifests	Transaction log (`_delta_log`, JSON + checkpoints)	Timeline of commits + file groups
Strength	Engine neutrality, hidden partitioning, partition evolution, REST catalog standard	Tight Spark / Databricks integration	Upsert-heavy and incremental processing
Typical stack	S3 + Glue / REST + Spark, Flink, Trino, Athena, Snowflake	Databricks, Spark	Spark, Flink on Hadoop / S3

All three solve the same problem (ACID tables on object storage). Iceberg is usually chosen when **several engines** must read and write the same tables without depending on one vendor.

Summary of what Iceberg gives on top of S3:

Capability	**How**
ACID transactions	Immutable files + atomic compare-and-swap of the metadata pointer in the catalog
Updates / deletes	`MERGE`, `UPDATE`, `DELETE` via copy-on-write or merge-on-read delete files
Fast queries	File-level statistics in manifests; no S3 listing; partition + min / max pruning
Time travel / rollback	Every commit is a snapshot F_k; old snapshots stay readable until expired
Schema evolution	Columns tracked by ID; changes are metadata-only
Partition evolution	Hidden partitioning with transforms; spec can change without rewriting data
Multi-engine	Open spec; Spark, Flink, Trino, Athena, Snowflake, DuckDB read the same table

Apache Iceberg turns a folder of Parquet files in S3 into a real table. The catalog holds a pointer to the current metadata; the metadata lists exactly which files form each snapshot. That single idea gives ACID commits, row-level updates for transactional (CDC) data, fast planning without listing S3, time travel, and schema / partition evolution, while any engine can query the same data. In exchange, tables need regular maintenance: compaction, snapshot expiration and orphan-file cleanup.
